

ادامه جزوه مرور CI/CD

بخش دوم - مطالب مطالعه شده بعد از فایل CI-CD-01

هدف این فایل: فقط ادامه مباحث بعد از فایل قبلی؛ بدون تکرار مفاهیم پایه‌ای که در CI-CD-01 آمده بود.

در فایل قبلی، مباحث تا Environment-specific Variables، نمونه Pipeline و مرور مفاهیم پایه جمع‌بندی شده بود. در انتهای همان فایل نیز موضوعاتی مثل Cache عملی، dependencies، ارتباط needs با artifacts، ساخت و Push کردن Docker Image، واقعی و Rollback به عنوان ادامه مسیر مشخص شده بودند. این فایل همان ادامه‌ای است که بعداً در گفت‌وگو مطالعه کردیم.

۱. پیکربندی عملی Cache برای NuGet

در بخش اول فقط تفاوت Cache و Artifact را یاد گرفته بودیم. بعداً دیدیم که برای Cache کردن Package‌های NuGet بهتر است مسیر Package‌ها را داخل فضای پروژه مشخص کنیم تا GitLab بتواند آن را بین Pipeline‌ها نگه دارد.

```
variables:
  NUGET_PACKAGES: "$CI_PROJECT_DIR/.nuget/packages"

cache:
  paths:
    - .nuget/packages/
```

در اجرای اول، دستور Package restore dotnet را دانلود می‌کند و Cache ذخیره می‌شود. در اجراهای بعدی، GitLab Cache را بازیابی می‌کند و بخش زیادی از Package‌ها دیگر لازم نیست از ابتدا دانلود شوند.

نکته مرور: Cache -> NuGet packages | خروجی Artifact -> publish

۲. مفهوم dependencies برای Artifact‌ها

مفهوم dependencies مشخص می‌کند یک Job به Artifact کدام Job‌های قبلی نیاز دارد. یعنی با آن تعیین می‌کنیم کدام خروجی قبلی در اختیار Job فعلی قرار بگیرد.

```
publish:
  stage: publish
  script:
```

```

- dotnet publish -c Release -o publish
artifacts:
  paths:
    - publish/

deploy:
  stage: deploy
  dependencies:
    - publish
  script:
    - echo "Deploying..."

```

در مثال بالا، Job مربوط به deploy خروجی Artifact مربوط به publish را دریافت می‌کند.

۳. دریافت Artifact با `needs:artifacts`

بعداً یاد گرفتیم که `needs` فقط برای ترتیب اجرا نیست و می‌تواند همراه Artifact هم استفاده شود. با `artifacts:` `true` می‌گوییم Artifact همان Job تعریف شده در `needs` نیز دریافت شود.

```

deploy:
  stage: deploy
  needs:
    - job: publish
      artifacts: true
  script:
    - echo "Deploy..."

```

در این حالت، `deploy` هم از نظر اجرا به `publish` وابسته است و هم Artifact آن را دریافت می‌کند.

جمع‌بندی: `needs` -> وابستگی اجرایی | `artifacts: true` -> دریافت خروجی همان Job

در ادامه نیز یاد گرفتیم بهتر است روی یک `needs`، Job و `dependencies` را بدون دلیل با هم ترکیب نکنیم؛ چون `needs` خودش می‌تواند دریافت Artifact های Job های مورد نیاز را مدیریت کند.

۴. ساخت Docker Image داخل Pipeline

بعد از Build و Test، وارد بخش عملی Docker شدیم. با دستور `docker build` از روی Dockerfile یک Image می‌سازیم.

```
docker build -t payment-api:1.0 .
```

در این دستور، `payment-api` نام Image و `Tag 1.0` آن است. Runner باید امکان اجرای Docker را داشته باشد.

برای نمونه، در سناریوی Docker-in-Docker می‌توان Job را داخل Docker CLI اجرا کرد و یک Docker daemon کمکی نیز در `services` قرار داد.

```
docker-build:
  stage: build
  image: docker:24.0.5-cli
  services:
    - docker:24.0.5-dind
  script:
    - docker build -t payment-api:1.0 .
```

جریان اصلی: **Dockerfile -> docker build -> Docker Image**

۵. Push کردن Image به Container Registry

اگر Image فقط روی Runner باقی بماند، Jobهای بعدی یا Server مقصد لزوماً به آن دسترسی ندارند. برای همین Image را در یک Container Registry مرکزی نگه می‌داریم.

```
script:
  - echo "$CI_REGISTRY_PASSWORD" | docker login "$CI_REGISTRY" -u "$CI_REGISTRY_USER" --
    password-stdin
  - docker build -t "$CI_REGISTRY_IMAGE:$CI_COMMIT_SHA" .
  - docker push "$CI_REGISTRY_IMAGE:$CI_COMMIT_SHA"
```

مفهوم `docker login` برای ورود به Registry است، `docker build` Image را می‌سازد و `docker push` همان Image را به Registry ارسال می‌کند.

دلیل **Registry: Image** از محیط موقت **Runner** خارج می‌شود و **Server** یا **Environment** مقصد می‌تواند آن را دریافت کند.

۶. Tag کردن Image با Commit SHA

بعداً یاد گرفتیم استفاده دائمی از `latest` مناسب ردیابی نسخه نیست؛ چون دقیقاً مشخص نمی‌کند کدام Commit روی Production اجرا شده است.

```
docker build -t payment-api:$CI_COMMIT_SHA .
```

با استفاده از `CI_COMMIT_SHA`، هر Image به Commit مشخصی وصل می‌شود. این موضوع هم برای **Traceability** و هم برای **Rollback** مهم است.

```
Commit: a81f96c
      |
      v
Image: payment-api:a81f96c
      |
```

مزیت **Tag** مشخص: می‌دانیم دقیقاً چه نسخه‌ای **Deploy** شده و نسخه قبلی را راحت‌تر پیدا می‌کنیم.

۷. **Deploy** کردن همان **Image** تست‌شده

در مرحله **Deploy**، بهتر است همان **Image** را اجرا کنیم که در **CI** ساخته و تست شده است؛ نه اینکه روی **Server** دوباره از **Source Code** یک **Image** جدید بسازیم.

نمونه روی یک **Server** ساده

```
docker pull registry.example.com/payment-api:a81f96c
docker run registry.example.com/payment-api:a81f96c
```

نمونه در **Kubernetes**

```
kubectl set image deployment/payment-api \
payment-api=registry.example.com/payment-api:a81f96c
```

ایده اصلی این است که **Image** **CI** را می‌سازد و **Push** می‌کند و **CD** همان **Image** تأییدشده را روی **Environment** مقصد **Deploy** می‌کند.

اصل مهم: چیزی را **Deploy** کن که قبلاً **Build** و **Test** کرده‌ای.

۸. **Rollback** با **Image** نسخه قبلی

اگر نسخه جدید بعد از **Deploy** مشکل داشته باشد، می‌توانیم **Image** نسخه سالم قبلی را دوباره **Deploy** کنیم. این همان **Rollback** است.

```
نسخه جدید: payment-api:a81f96c
نسخه قبلی: payment-api:73bc210

kubectl set image deployment/payment-api \
payment-api=registry.example.com/payment-api:73bc210
```

به همین دلیل نگه داشتن **Tag**های دقیق اهمیت زیادی دارد؛ چون نسخه قبلی سالم را سریع پیدا می‌کنیم.

۹. استفاده از همان **Image** در **Development**، **Staging** و **Production**

یکی از نکات مهمی که بعداً یاد گرفتیم این بود که بهتر است برای هر Environment دوباره Image نسازیم. همان Image باید از Development به Staging و سپس Production منتقل شود.

```
payment-api:a81f96c
|
+--> Development
|
+--> Staging
|
+--> Production
```

این کار باعث می‌شود نسخه‌ای که در Staging تست شده دقیقاً همان نسخه‌ای باشد که وارد Production می‌شود.

اصل معماری: یک بار Build کن و همان Artifact یا Image را بین Environment ها Promote کن.

۱۰. تکمیل rules با سناریوی Branch ها

در فایل اول با rules آشنا شده بودیم. بعداً آن را در یک سناریوی واقعی‌تر استفاده کردیم: Branch مربوط به develop برای Development و Branch مربوط به main برای Production.

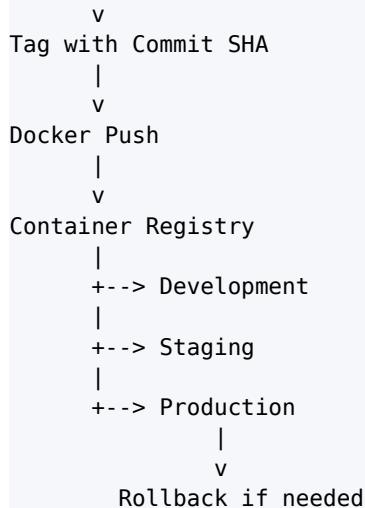
```
deploy-development:
  stage: deploy
  script:
    - echo "Deploy Development"
  rules:
    - if: '$CI_COMMIT_BRANCH == "develop"'

deploy-production:
  stage: deploy
  script:
    - echo "Deploy Production"
  rules:
    - if: '$CI_COMMIT_BRANCH == "main"'
  when: manual
```

در این نمونه، Job مربوط به Development فقط روی develop وارد Pipeline می‌شود و Job مربوط به Production فقط روی main قرار می‌گیرد. همچنین Deploy به Production به صورت دستی نگه داشته شده است.

۱۱. Flow کامل مطالب بخش دوم

```
Developer Push
|
v
Restore / Build / Test
|
v
Docker Build
|
```



۱۲. مرور خیلی سریع برای مصاحبه

مفهوم	خلاصه
GitLab در Cache	برای جلوگیری از دانلود یا کار تکراری؛ نمونه رایج آن Package های NuGet است.
dependencies	مشخص می کند Artifact کدام Job های قبلی مورد نیاز است.
needs: artifacts	وابستگی Job را تعریف می کند و در صورت artifacts: true خروجی همان Job را نیز می گیرد.
docker build	از روی Dockerfile یک Docker Image می سازد.
Container Registry	محل مرکزی نگهداری Image ها برای دسترسی Job ها و Server های مقصد است.
CI_COMMIT_SHA	برای Tag یکتا و ردیابی دقیق نسخه Image استفاده می شود.
Deploy	همان Image ساخته و تست شده را روی Environment مقصد اجرا می کند.
Rollback	نسخه خراب را با Image سالم قبلی جایگزین می کند.
Promote Image	همان Image را از Development به Staging و Production می بریم.
rules	اجرای Job را بر اساس شرطهایی مثل Branch کنترل می کند.

سؤال: چرا **Docker Image** را **Push** می‌کنیم؟

پاسخ: چون **Image** روی **Runner** محلی و موقتی است و **Registry** آن را در دسترس **Job**ها و **Server**های مقصد قرار می‌دهد.

سؤال: چرا **Tag** مبتنی بر **Commit** بهتر از **latest** است؟

پاسخ: چون دقیقاً مشخص می‌کند چه **Commit**ی **Deploy** شده و **Rollback** را ساده‌تر می‌کند.

سؤال: چرا روی **Server** دوباره **Image** نمی‌سازیم؟

پاسخ: چون باید همان **Image**ی را **Deploy** کنیم که در **CI** ساخته و تست شده است.

سؤال: چرا همان **Image** را در همه **Environment**ها استفاده می‌کنیم؟

پاسخ: چون می‌خواهیم نسخه‌ای که در **Staging** تست شده دقیقاً همان نسخه **Production** باشد.

سؤال: **Rollback** چیست؟

پاسخ: برگرداندن برنامه به نسخه سالم قبلی، معمولاً با **Deploy** دوباره **Image** قبلی.

وضعیت مسیر: برای مصاحبه **Senior .NET Developer**، مباحث اصلی **CI/CD** تا اینجا کافی هستند؛ ادامه عمیق‌تر بیشتر وارد جزئیات **DevOps** می‌شود.